

MongoDB

Experiment-05

05. Execute Aggregation operations (\$avg, \$min,\$max, \$push, \$addToSet etc. students encourage to execute several queries to demonstrate various aggregation operators)

Aggregation Framework in MongoDB

The aggregation framework in MongoDB is a powerful tool that enables you to process and manipulate data stored in collections.

It allows you to perform various operations like filtering, grouping, sorting, and calculations on your documents to generate summarized and insightful results.

Key Aggregation Operators

1. \$avg (Calculate Average):

Calculates the average value of a specified field across all documents in a group.

Example:

```
db.students.aggregate([
  { $group: { _id: "$subject", averageScore: { $avg: "$score" } } }
]);
```

This pipeline groups students by their subject (`\$subject`) and calculates the average score (`\$avg: "\$score"`) for each subject group.

Example 01:

Finding all the student details.

```
mongosh mongodb://127.0.0.1:27017
db> db.student1.find();
[
  {
    _id: ObjectId('66645d86df4bc5f5cdf985e0'),
    name: 'Student 948',
    age: 19,
    courses: "['English', 'Computer Science', 'Physics', 'Mathematics']",
    gpa: 3.44,
    home_city: 'City 2',
    blood_group: 'O+',
    is_hotel_resident: true
  },
  {
    _id: ObjectId('66645d86df4bc5f5cdf985e1'),
    name: 'Student 157',
    age: 20,
    courses: "['Physics', 'English']",
    gpa: 2.77,
    home_city: 'City 4',
    blood_group: 'O-',
    is_hotel_resident: true
  },
  {
    _id: ObjectId('66645d86df4bc5f5cdf985e2'),
    name: 'Student 316',
    age: 20,
    courses: "['Physics', 'Computer Science', 'Mathematics', 'History']",
    gpa: 2.82,
    blood_group: 'B+',
    is_hotel_resident: true
  }
]
```

Example 02:

This pipeline groups students by their `_id` and calculates the average gpa (``$avg: "$gpa"``).

```
db> db.student1.aggregate([
...   {$group: {_id: null, averageGPA: {$avg: "$gpa"}}}
... ]);
```

Output:

```
[ { _id: null, averageGPA: 3.2268699186991867 } ]
```

Example 03:

This pipeline groups students by their `_id` is home city and calculates the average gpa (``$avg: "$gpa"``).

```
db> db.student1.aggregate([ { $group: { _id: "$home_city", averageGPA: { $avg: "$gpa" } } }
] );
[
  { _id: 'City 6', averageGPA: 3.173478260869565 },
  { _id: null, averageGPA: 3.2426027397260273 },
  { _id: 'City 8', averageGPA: 3.329285714285714 },
  { _id: 'City 2', averageGPA: 3.2945 },
  { _id: 'City 10', averageGPA: 3.147 },
  { _id: 'City 1', averageGPA: 3.21 },
  { _id: 'City 9', averageGPA: 3.326842105263158 },
  { _id: 'City 7', averageGPA: 3.184 },
  { _id: 'City 5', averageGPA: 3.274736842105263 },
  { _id: 'City 4', averageGPA: 2.9992857142857146 },
  { _id: 'City 3', averageGPA: 3.2242105263157894 }
]
```

Example 04:

This pipeline groups students by their `_id` is blood group and calculates the average gpa (``$avg: "$gpa"``).

```
db> db.student1.aggregate([ { $group: { _id: "$blood_group", averageGPA: { $avg: "$gpa" } }
} ] );
[
  { _id: null, averageGPA: 3.188823529411765 },
  { _id: 'B+', averageGPA: 3.2724137931034485 },
  { _id: 'AB+', averageGPA: 3.2870833333333334 },
  { _id: 'O-', averageGPA: 3.1703225806451614 },
  { _id: 'B-', averageGPA: 3.165483870967742 },
  { _id: 'A-', averageGPA: 3.2963999999999998 },
  { _id: 'O+', averageGPA: 3.2024137931034486 },
  { _id: 'A+', averageGPA: 3.2206666666666667 },
  { _id: 'AB-', averageGPA: 3.25 }
]
```

2. \$min (Find Minimum Value):

Returns the minimum value of a field within a group.

Example:

```
db.students.aggregate([
  { $group: { _id: "$subject", lowestScore: { $min: "$score" } } }
]);
```

This pipeline groups students by subject and finds the lowest score (`\$min: "\$score"`) for each subject.

3. \$max (Find Maximum Value):

Returns the maximum value of a field within a group.

Example:

```
db.students.aggregate([
  { $group: { _id: "$subject", highestScore: { $max: "$score" } } }
])
```

This pipeline groups students by subject and finds the highest score (`\$max: "\$score"`) for each subject.

Example 05:

Finds the minimum age and maximum age from the student1 db.

```
db> db.student1.aggregate([ {$group:{_id:null,minAge:{$min:"$age"},maxAge:{$max:"$age"}}}])
;
[ { _id: null, minAge: 18, maxAge: 25 } ]
```

4. \$push (Include Multiple Values):

Adds an element to an array field in each document within a group.

Example:

```
db.students.aggregate([
  { $match: { score: { $gt: 90 } } }, // Filter for scores above 90
  { $group: { _id: null, topScorers: { $push: "$name" } } }
])
```

This pipeline filters for students with scores above 90 (`\$match`) and then groups all their names (`\$push: "\$name"`) into a single array named `topScorers`.

Example 06:

This pipeline groups all the students with their courses (`\$push: "\$courses"`) into a single array named `allCourses`.

```
db> db.student1.aggregate([{$group: { _id: 0, allCourses: { $push: "$courses" } }
}]);
[
  {
    _id: 0,
    allCourses: [
      ["English", "Computer Science", "Physics", "Mathematics"],
      ["Physics", "English"],
      ["Physics", "Computer Science", "Mathematics", "History"],
      ["Mathematics", "History", "English"],
      ["English", "Computer Science", "Mathematics", "History"],
      ["History", "Physics", "Computer Science", "Mathematics"],
      ["History", "Physics", "Computer Science"],
      ["Computer Science", "Mathematics", "History", "English"],
      ["Mathematics", "Computer Science", "Physics"],
      ["History", "Physics", "Computer Science"],
      ["English", "History"],
      ["History", "Physics", "Mathematics"],
      ["English", "Physics"],
      ["Computer Science", "Physics", "History", "Mathematics"],
      ["Physics", "Computer Science", "English"],
      ["Computer Science", "Physics", "Mathematics", "History"],
      ["Physics", "Computer Science", "English", "Mathematics"],
      ["Computer Science", "History", "Physics", "English"],
      ["History", "Physics", "Computer Science"],
      ["History", "Mathematics", "Physics", "English"],
      ["Computer Science", "Mathematics", "English", "Physics"],
      ["English", "Mathematics", "Computer Science"],
      ["English", "History"],
      ["Mathematics", "History", "Physics"],
      ["History", "Computer Science"],
      ["English", "Physics", "Mathematics", "Computer Science"],
      ["English", "Computer Science", "Physics"],
      ["English", "History", "Computer Science"],
      ["English", "History", "Mathematics", "Computer Science"],
      ["English", "History", "Mathematics", "Computer Science"]
    ]
  }
]
```

5. \$addToSet (Include Unique Values):

Adds a unique element to an array field in each document within a group.

Example:

```
db.students.aggregate([
  { $group: { _id: "$subject", enrolledStudents: { $addToSet:
"$name" } } }
])
```

This pipeline groups students by subject and creates an array named `enrolledStudents` containing unique student names (`\$addToSet: "\$name"`) for each subject.

Example 07:

```
db> db.candid.aggregate([ { $unwind: "$courses" }, { $group: { _id: null, uniqueCo
urses: { $addToSet: "$courses" } } } ] );
[
  {
    _id: null,
    uniqueCourses: [
      'Marine Science',
      'Music History',
      'History',
      'Mathematics',
      'Psychology',
      'Philosophy',
      'Sociology',
      'Chemistry',
      'Political Science',
      'Robotics',
      'Environmental Science',
      'Literature',
      'English',
      'Physics',
      'Biology',
      'Ecology',
      'Artificial Intelligence',
      'Cybersecurity',
      'Art History',
      'Creative Writing',
      'Engineering',
      'Computer Science',
      'Statistics',
      'Film Studies'
    ]
  }
]
db> |
```

This pipeline groups students containing unique student courses (`$addToSet: "$courses"`) for each courses.

Aggregation operations

Operator	Meaning
<code>\$count</code>	Calculates the quantity of documents in the given group.
<code>\$max</code>	Displays the maximum value of a document's field in the collection.
<code>\$min</code>	Displays the minimum value of a document's field in the collection.
<code>\$avg</code>	Displays the average value of a document's field in the collection.
<code>\$sum</code>	Sums up the specified values of all documents in the collection.
<code>\$push</code>	Adds extra values into the array of the resulting document.

- Aggregation pipelines consist of multiple stages, each performing a specific operation on the data.
- We can chain these stages to achieve complex data transformations and calculations.
- The `$group` operator is essential for grouping documents and performing calculations on grouped data.

Benefits of Aggregation:

➤ Data Summarization:

Aggregation helps you condense large datasets into meaningful summaries, enabling easier analysis and visualization.

➤ **Improved Efficiency:**

By performing computations within the database, aggregation avoids transferring large amounts of raw data to the client, enhancing performance.

➤ **Flexibility:**

The modular nature of the pipeline allows you to combine multiple operators to achieve complex data manipulations.

➤ **Scalability:**

Aggregation pipelines can efficiently handle large datasets, making them suitable for big data scenarios.